Project 2 – Acme Analytics Linux Platform

Day 5 · Phase 2 complete: NFS access test and LVM snapshot rollback

Muhammad Bilal Javed · 20 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	2 – Storage, sharing and recovery
Hosts worked on	gw01 (access test), stor01 (LVM snapshot, NFS server), app01 (NFS client)
Status	Phase 2 ✅ – Steps 2.1–2.4 done; VMware snapshots `30-storage-*`
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 4.

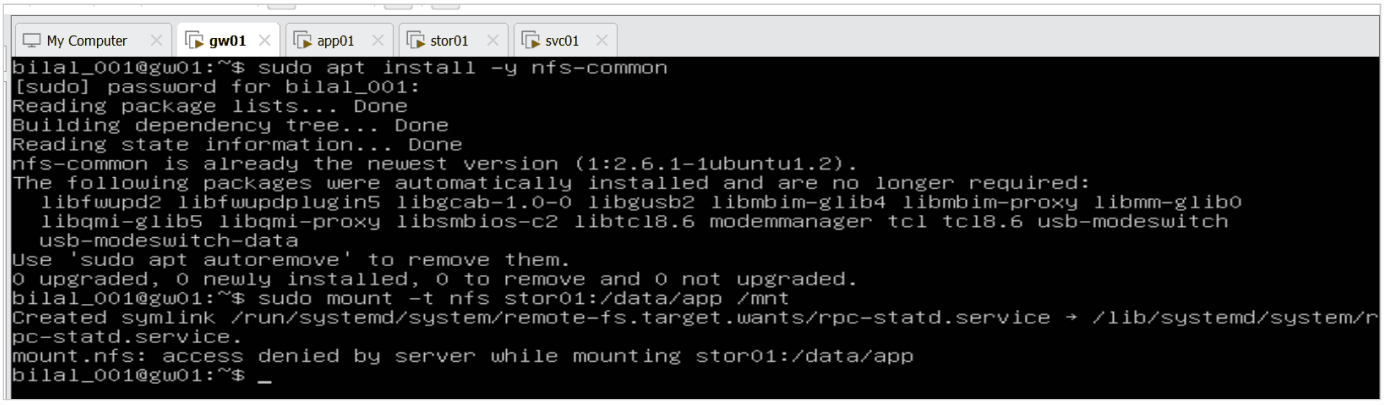
---

1. Break test – reachable is not allowed (Step 2.2)

💡 **In plain words:** gw01 has stor01 in `/etc/hosts` and can ping it – but it is not listed in stor01's `/etc/exports`. Can it use the share anyway?

```
# on gw01
sudo apt install -y nfs-common                  # NFS client tools (already installed)
sudo mount -t nfs stor01:/data/app /mnt         # /mnt = standard empty folder for temporary mounts
```

📸 **Evidence – `access denied by server`:**



access denied

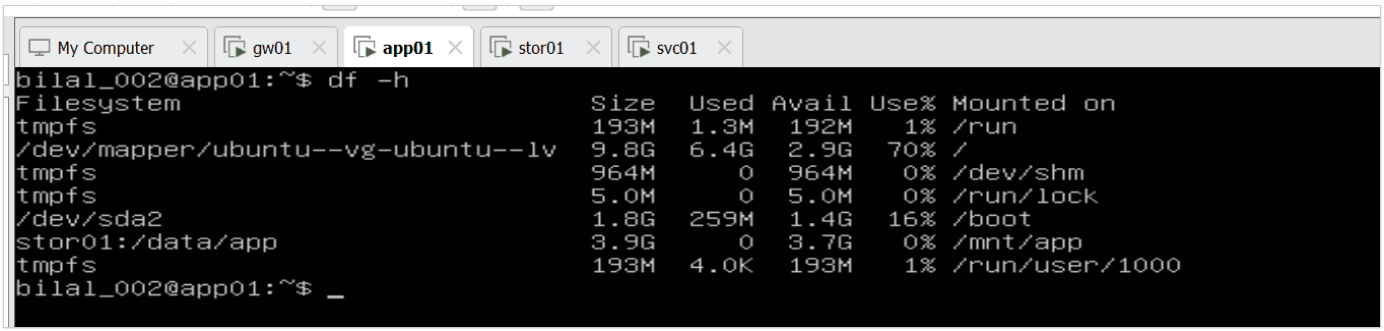
Lesson: being able to reach a server is not permission to use its shares. Only the IPs listed in `/etc/exports` (here only app01, 192.168.60.10) may mount `/data/app`.

2. Where the app's data really lives

```
app01: /mnt/app  ═ NFS ═▶  stor01: /data/app  ─inside─▶  /data   ─mounted from─▶  lvdata (logical volume)
                                                  /backups ─mounted from─▶  lvbackups
```

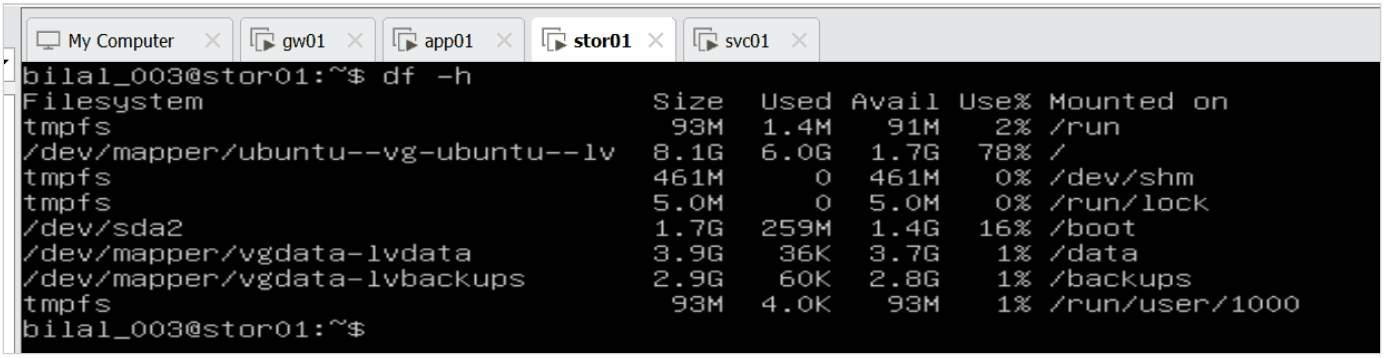
- `/mnt/app` is a folder on **app01**; `/data/app` is a folder on **stor01**. Over NFS, everything done in `/mnt/app` really happens in `/data/app`.
- On stor01 the logical volume **`lvdata`** is mounted on `/data` – so every file in `/data/app` is stored on `lvdata`. **The work happens on `lvdata`; the backups are on another volume, `lvbackups`.**

📸 **Evidence – app01: `stor01:/data/app` on `/mnt/app`:**



app01 df

📸 **Evidence – stor01: `lvdata` on `/data`, `lvbackups` on `/backups`:**



stor01 df

3. LVM snapshot – the undo button (Step 2.4)

💡 **The story:** a risky change is coming to `/data` (an update, a cleanup). A mistake made from app01 lands directly on `lvdata`, and the rsync mirror – which keeps only the current state – would copy it at its next run. The dated versions still hold good files, but restoring file by file is slow. So **before** the change: take a snapshot – a frozen state of the whole volume – and roll back in seconds if it goes wrong.

3.1 Take the snapshot, make the "mistakes"

```
# on app01 — release the share first (see the issue below)
sudo umount /mnt/app

# on stor01
ls /data/app                                          # lab.txt new.txt test.txt
sudo lvcreate -s -n snap1 -L 1G /dev/vgdata/lvdata    # -s snapshot of lvdata; 1 GB room for changes
sudo lvs
echo "bad change" | sudo tee /data/app/oops.txt       # mistake 1
sudo rm /data/app/test.txt                            # mistake 2
ls /data/app                                          # lab.txt new.txt oops.txt
```

`lvs` shows	Meaning
`lvdata  owi-aos---`	**o** = origin: a snapshot is watching this volume
`snap1  swi-a-s---  1.00g  Origin lvdata`	**s** = snapshot of lvdata, 1 GB room for changes
`Data% 0.01`	How full the snapshot's room is – at 100 % it becomes invalid

The mistakes were made on stor01 for simplicity; from app01 through `/mnt/app` the effect is identical – it is the same file on `lvdata`, and the snapshot protects the volume, whoever made the change.

📸 **Evidence:**

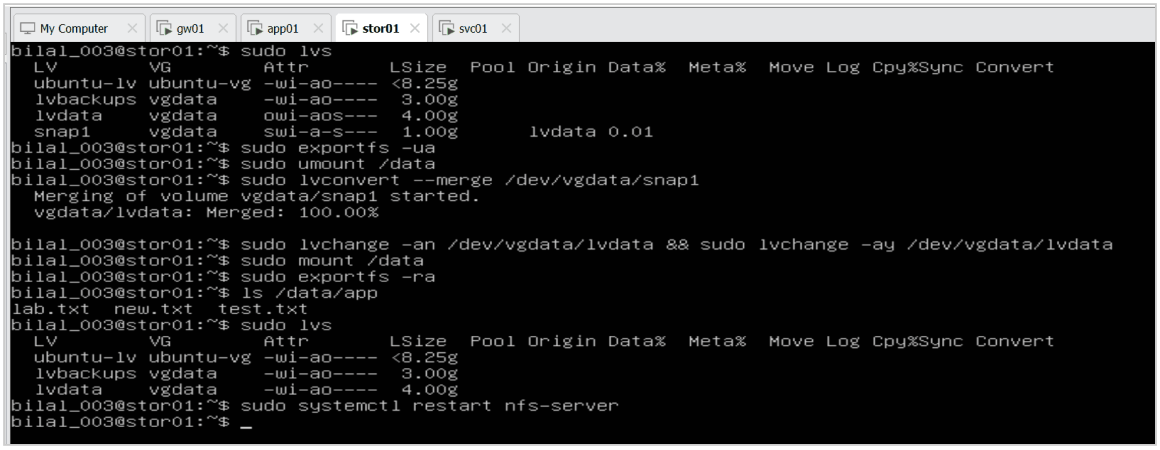
```
bilal_003@stor01:~$ ls /data/app
lab.txt  new.txt  test.txt
bilal_003@stor01:~$ sudo lvcreate -s -n snap1 -L 1G /dev/vgdata/lvdata
[sudo] password for bilal_003:
  Logical volume "snap1" created.
bilal_003@stor01:~$ sudo lvs
  LV        VG        Attr       LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  ubuntu-lv ubuntu-vg -wi-ao---- <8.25g
  lvbackups vgdata    -wi-ao----  3.00g
  lvdata    vgdata    owi-aos---  4.00g
  snap1     vgdata    swi-a-s---  1.00g      lvdata 0.01
bilal_003@stor01:~$ echo "bad change" | sudo tee /data/app/oops.txt
bad change
bilal_003@stor01:~$ sudo rm /data/app/test.txt
bilal_003@stor01:~$ ls /data/app
lab.txt  new.txt  oops.txt
bilal_003@stor01:~$ _
```

snapshot taken

3.2 Roll back

```
# on stor01
sudo exportfs -ua                                     # stop sharing /data/app
sudo umount /data                                     # nobody may use the volume during the merge
sudo lvconvert --merge /dev/vgdata/snap1              # back to the frozen state
sudo lvchange -an /dev/vgdata/lvdata && sudo lvchange -ay /dev/vgdata/lvdata   # deactivate / activate
sudo mount /data
sudo exportfs -ra                                     # share again
ls /data/app                                          # lab.txt new.txt test.txt — oops.txt gone
sudo lvs                                              # snap1 gone — merged into lvdata
```

📸 **Evidence – `Merged: 100.00%`; test.txt back, oops.txt gone; snap1 gone:**

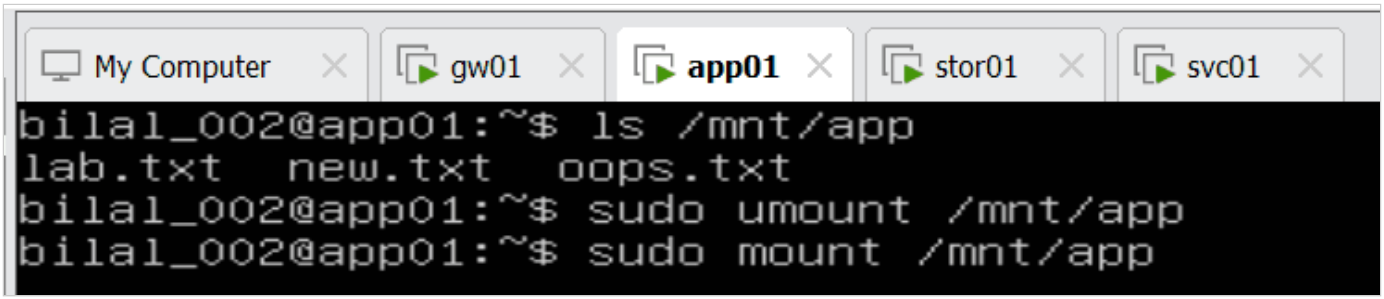


rollback

3.3 Issue – app01 hangs after the rollback (stale NFS mount)

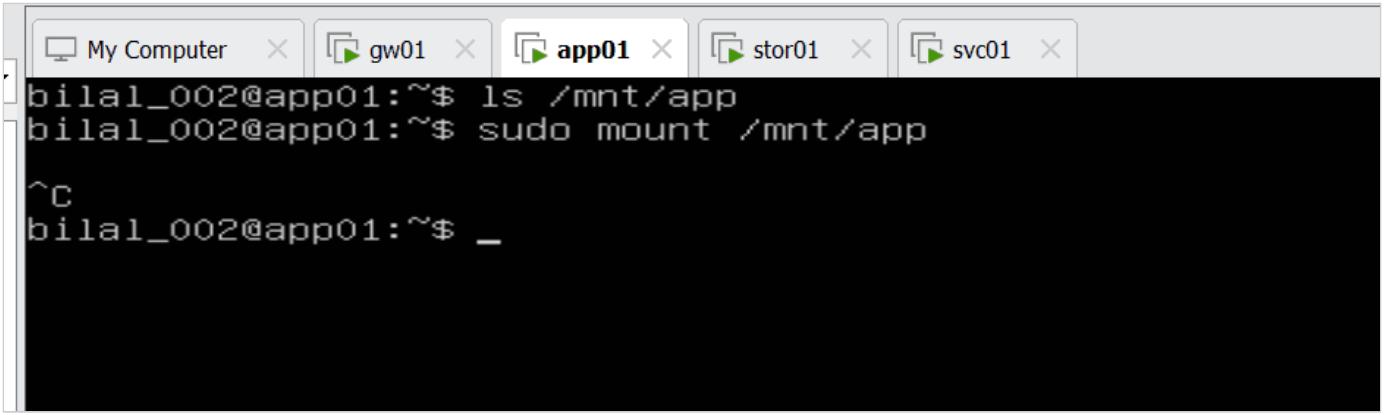
Seen	After the rollback, `mount /mnt/app` on app01 hung and had to be stopped with Ctrl+C.
Cause	app01 still had the share mounted **during** the rollback – `ls /mnt/app` still showed `oops.txt`; it was unmounted only afterwards. stor01 pulled the volume away under a connected client, which was left with an old, invalid connection (a **stale** mount).
Fix	stor01: `sudo systemctl restart nfs-server` · app01: `sudo umount -f -l /mnt/app` (force, lazy), `showmount -e stor01`, `sudo mount -a`
Lesson	Before rolling back a volume shared over NFS, **unmount every client first** – the client lets go, then the server.

📸 **Evidence – app01 still mounted during the rollback (oops.txt visible):**



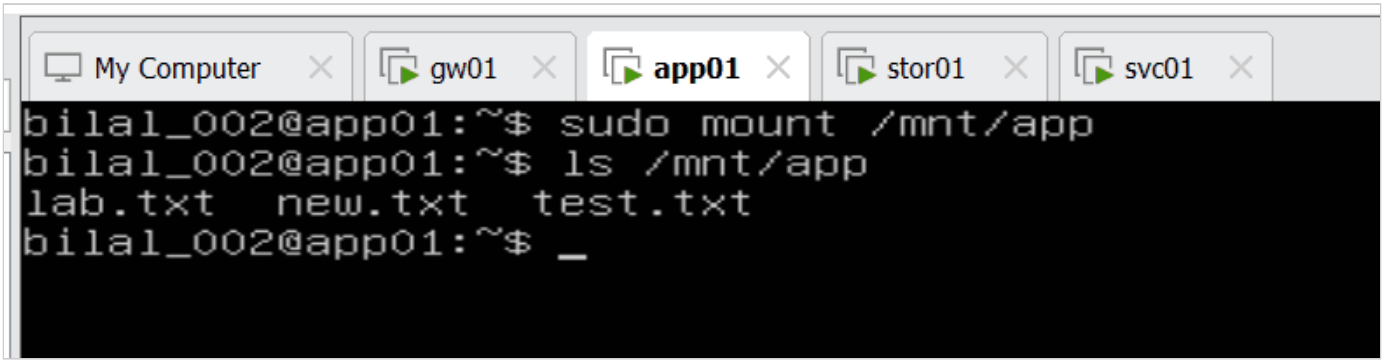
still mounted

📸 **Evidence – the mount hangs:**



hang

📸 **Evidence – after the fix, app01 sees the rolled-back files:**



fixed

4. Snapshot vs backup

	LVM snapshot	Backup (versions in `/backups`)
What	A **frozen state of the whole volume** at the moment it was taken	**Separate copies** of the files
How	Keeps only the old version of blocks that change afterwards (copy-on-write), in its small room	Full files (unchanged ones hard-linked)
Where	**Same disks** as `lvdata`	Separate volume `lvbackups` – ideally another server
Restore	The **whole volume at once**, in seconds	Files **one by one**, from any day
Survives a reboot	Yes	Yes
Lost when	Disks die, its room fills to 100 %, merged or removed	Deleted, or the disks die – hence copies off-site

Snapshot = quick undo, short-term, same disks. Backup = recovery, long-term, separate volume.

5. VMware snapshots

VM	Snapshot	Description
gw01	`30-storage-gw01`	End of Phase 2; NFS client tools for the access test
app01	`30-storage-app01`	NFS client: stor01:/data/app on /mnt/app (fstab, `_netdev,nofail`)
stor01	`30-storage-stor01`	LVM vgdata (lvdata /data 4 GB, lvbackups / backups 3 GB), NFS server, tar/rsync backups v1–v3
svc01	`30-storage-svc01`	GLPI; restore ticket closed

Key understanding (in my own words)

> On app01, `/mnt/app` is a folder. On stor01, `/data/app` is a folder. They are connected over NFS, so everything done in `/mnt/app` on app01 really happens in `/data/app` on stor01. On stor01, the logical volume `lvdata` is mounted on `/data`, so every file in `/data`, including `/data/app`, is stored on `lvdata`. The work happens on `lvdata`; the backups are stored on another logical volume, `lvbackups`, mounted on `/backups`.
>
> If someone on app01 makes a wrong or risky change in `/mnt/app`, the mistake lands directly on `lvdata`. The rsync mirror only keeps the current state, so at its next run it would copy the mistake too. A backup is a set of separate copies of the files on `lvbackups`: from it I can restore files one by one, from any day, but that is slow for a big mistake.
>
> So before a risky change I take an **LVM snapshot**: a frozen state of the whole `lvdata` volume, taken at the moment I create it. If the change goes wrong, I roll back the whole volume at once. To roll back, nobody may be using the volume: first app01 unmounts `/mnt/app`, then stor01 stops sharing (`exportfs -ua`) and unmounts `/data`. I learned this the hard way – app01 was still mounted during my rollback, was left with a stale mount and hung, until I restarted the NFS server and force-unmounted the client.
>
> Both survive a reboot, but a snapshot lives on the same disks and becomes invalid if its room fills up: **snapshot = quick undo, short-term, same disks; backup = recovery, long-term, separate volume.** And gw01 showed that being able to reach a server is not permission to use its shares.

6. Next steps

1. Phase 2 tickets in GLPI: LVM build (Change), grow /data (Request), fstab UUID typo (Incident), NFS share (Change), stale NFS mount after rollback (Incident).
2. **Phase 3 – automation:** systemd services and timers – the nightly backup runs by itself.